МИНИСТЕРСТВО ЦИФРОВЫХ ТЕХНОЛОГИЙ РЕСПУБЛИКИ УЗБЕКИСТАН
ТАШКЕНТСКИЙ УНИВЕРСИТЕТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ ИМЕНИ МУХАММАДА АЛ-ХОРАЗМИЙ

Дисциплина: «Введение в программную инженерию»

ПРАКТИЧЕСКАЯ РАБОТА № 11

Тема 11. Тестирование программного обеспечения на уровне компонентов: тест-план и тест-кейсы

Вариант № 8
«Разработка умного сайта для заказа услуг»

Выполнила: студентка группы М320-24 ДИР
Камбарова Хатича

Проверила: Латипова Нодира

Ташкент — 2026

1. Тема практической работы

Тестирование программного обеспечения на уровне компонентов: тест-план и тест-кейсы.

Цель работы: закрепить теоретические знания; научиться планировать тестирование компонентов программного обеспечения, разрабатывать тест-кейсы методами «чёрного ящика» (эквивалентное разбиение, анализ граничных значений) и «белого ящика» (тестирование базисного пути), применять драйверы и заглушки, вести протокол тестирования и оформлять отчёты о дефектах.

Вариант № 8: «Разработка умного сайта для заказа услуг» («XizmatGo»). Объект тестирования — подсистема **SS-3 «Управление заказами»** из модели проектирования ПР-6 (диаграмма проектных классов). Требования взяты из ПР-5 (FR-04, FR-07, FR-08, UC-03, UC-08), требования к качеству — из ПР-9 (QR-03 «100 % эталонных расчётов цены верны», QR-04). Компоненты реализованы на Python 3, тесты — на **pytest**, потоковый граф построен в **draw.io** (файл *Kambarova_Xaticha_Variant8_PR11_potokovyi_graf.drawio*).

2. Тестируемые компоненты и тест-план

Таблица 2.1 — Тестируемые компоненты подсистемы SS-3

№	Компонент (класс ПР-6)	Назначение	Проверяемые требования	Метод тестирования
К-1	**PriceCalculator** (pricing.calculate)	Расчёт предварительной стоимости заказа по бизнес-правилам	FR-04, UC-03.1, QR-03	«Чёрный ящик»: эквивалентное разбиение и граничные значения
К-2	**OrderService.accept_order**	Принятие заказа мастером с проверками (4 условия)	FR-08, UC-08, FR-07 (событие для уведомления)	«Белый ящик»: базисные пути; заглушки зависимостей
К-3	**OrderStateMachine** (states.can_change)	Допустимые переходы статусов заказа по ролям	FR-07, UC-09, SR-04 (авторизация, ПР-10)	«Чёрный ящик»: таблица переходов (допустимые и недопустимые)

Таблица 2.2 — Тест-план компонентного тестирования

Объекты тестирования	Компоненты К-1, К-2, К-3 подсистемы SS-3, версия 0.1 (ветка feature/orders), изолированно от БД, сети и внешних систем
Проверяемые требования	FR-04 (формула цены: база × объём, скидка 10 % от 5 единиц, срочность +30 %, выезд 15 000 сум, интервал 08:00–21:00, объём 1–10); FR-08 / UC-08 (принять можно только существующий заказ в статусе NEW верифицированным мастером со свободным слотом); FR-07 (правила переходов статусов и публикация события); QR-03 (100 % эталонных расчётов цены)
Не тестируется	Интерфейс, реальные PostgreSQL, Redis, платёжные системы и SMS (это уровни интеграционного и системного тестирования); производительность (QR-08)
Подход	Модульное тестирование, разработка тестов до реализации (test-first): тест-кейсы составлены по требованиям, затем запущены на коде. К-1 и К-3 — «чёрный ящик», К-2 — «белый ящик» по базисным путям. Драйверы — тестовые функции pytest; зависимости К-2 заменены заглушками и шпионами. Автоматический прогон при каждом изменении в CI (план SQA ПР-9)

Окружение и средства	Python 3.13, pytest 9.1, параметризованные тесты (@pytest.mark.parametrize), фикстуры; заглушки FakeOrderRepository, StubMasterDirectory, StubSchedule; шпион SpyEventBus; Linux, GitHub Actions; учёт дефектов — GitHub Issues
Критерии начала	Компоненты собраны и импортируются без ошибок; требования FR-04, FR-08 утверждены владельцем продукта; тест-кейсы прошли ревью; окружение pytest развёрнуто
Критерии завершения	Выполнены 100 % запланированных тест-кейсов; пройдены 100 %; покрыты все классы эквивалентности, граничные значения и все 5 базисных путей; открытых дефектов S1–S2 нет; каждое требование связано хотя бы с одним тест-кейсом
Критерии приостановки	Компонент не импортируется или более 50 % тестов падают по одной причине — тестирование приостанавливается до исправления сборки
Роли (ПР-4)	Ким Д. (QA) — тест-план, тест-кейсы, протокол, отчёты о дефектах; Рахимов Ж. (backend) — реализация компонентов, заглушки, исправление дефектов, код-ревью тестов; Камбарова Х. (PO) — утверждение бизнес-правил и ожидаемых результатов
Сроки	Спринт 2 (5 рабочих дней): день 1 — тест-план и тест-кейсы; день 2–3 — реализация тестов и заглушек; день 4 — прогон, отчёты о дефектах, исправление; день 5 — повторный прогон, протокол, трассировка
Риски	Неоднозначность бизнес-правил («от 5» или «больше 5») — уточнение с PO до написания тестов; заглушки ведут себя не как реальные компоненты — контракты интерфейсов из ПР-6, проверка на интеграционном уровне; нехватка времени QA — приоритет тестам K-1 и K-2 (деньги и заказы)

3. Классы эквивалентности, граничные значения и тест-кейсы «чёрного ящика»

Компонент с бизнес-правилами — **K-1 PriceCalculator**. Сигнатура: `calculate(base_price, quantity, hour, urgent) → PriceRange(min, max)`. Правила (FR-04): стоимость работ = базовая цена × объём; при объёме от 5 единиц — скидка 10 % на работы; при «Срочно» — надбавка 30 % на работы; итог = работы + выезд 15 000 сум; верхняя граница диапазона = итог × 1,5, округлённая вверх до 1 000 сум. Объём — целое число 1…10; начало визита — с 08:00 до 21:00 (час 8…20).

Таблица 3.1 — Классы эквивалентности и граничные значения

Входное условие	Допустимые классы	Недопустимые классы	Граничные значения
Объём quantity	V1: 1 ≤ q ≤ 4 (без скидки) V2: 5 ≤ q ≤ 10 (скидка 10 %)	N1: q < 1 N2: q > 10 N3: не целое число	0, **1**, 4, **5**, **10**, 11
Час визита hour	V3: 8 ≤ h ≤ 20	N4: h < 8 N5: h ≥ 21	7, **8**, **20**, 21
Срочность urgent	V4: false (без надбавки) V5: true (+30 %)	—	—
Базовая цена base_price	V6: > 0	N6: ≤ 0	0, 1

Жирным выделены граничные значения, лежащие внутри допустимых классов.

Таблица 3.2 — Тест-кейсы «чёрного ящика» для PriceCalculator (base_price = 80 000 сум)

ID	Входные данные (q, h, urgent)	Покрываемые классы / границы	Ожидаемый результат (расчёт)	Результат
TC-P01	1, 8, нет	V1, V3, V4; q = 1, h = 8	80 000 + 15 000 = **95 000**; max ⌈142 500⌉ = **143 000**	PriceRange(95 000, 143 000)

TC-P02	2, 12, нет	V1, V3, V4 (типичное)	160 000 + 15 000 = **175 000**; max **263 000**	(175 000, 263 000)
TC-P03	4, 20, нет	V1; q = 4 (перед порогом), h = 20	320 000 + 15 000 = **335 000**; max **503 000**	(335 000, 503 000)
TC-P04	5, 12, нет	V2; q = 5 (порог скидки)	400 000 × 0,9 + 15 000 = **375 000**; max **563 000**	(375 000, 563 000)
TC-P05	10, 12, нет	V2; q = 10 (максимум)	800 000 × 0,9 + 15 000 = **735 000**; max **1 103 000**	(735 000, 1 103 000)
TC-P06	1, 12, да	V1, V5	80 000 × 1,3 + 15 000 = **119 000**; max **179 000**	(119 000, 179 000)
TC-P07	5, 12, да	V2, V5; q = 5	400 000 × 0,9 × 1,3 + 15 000 = **483 000**; max **725 000**	(483 000, 725 000)
TC-P08	0, 12, нет	N1; q = 0	Исключение PriceError «Объём должен быть от 1 до 10»	
TC-P09	11, 12, нет	N2; q = 11	Исключение PriceError	
TC-P10	2,5, 12, нет	N3	Исключение PriceError «Объём должен быть целым числом»	
TC-P11	1, 7, нет	N4; h = 7	Исключение PriceError «Мастер приезжает с 08:00 до 21:00»	
TC-P12	1, 21, нет	N5; h = 21	Исключение PriceError	
TC-P13	base = 0; 1, 12, нет	N6	Исключение PriceError «Базовая цена должна быть больше нуля»	

Каждый допустимый класс покрыт хотя бы одним тест-кейсом (несколько классов в одном кейсе), каждый недопустимый — отдельным кейсом, чтобы одна ошибка не скрывала другую. Компонент К-3 протестирован таблицей переходов: 4 допустимых (TC-S01…S04) и 4 недопустимых (TC-S05…S08) перехода (листинг 5.7).

4. Потоковый граф, цикломатическая сложность и тест-кейсы «белого ящика»

Функция **OrderService.accept_order(order_id, master_id)** (листинг 5.1, строки 31–46) содержит 4 условия: заказ не найден; статус не NEW; мастер не верифицирован; слот мастера занят. Номера узлов графа указаны в комментариях к строкам кода.

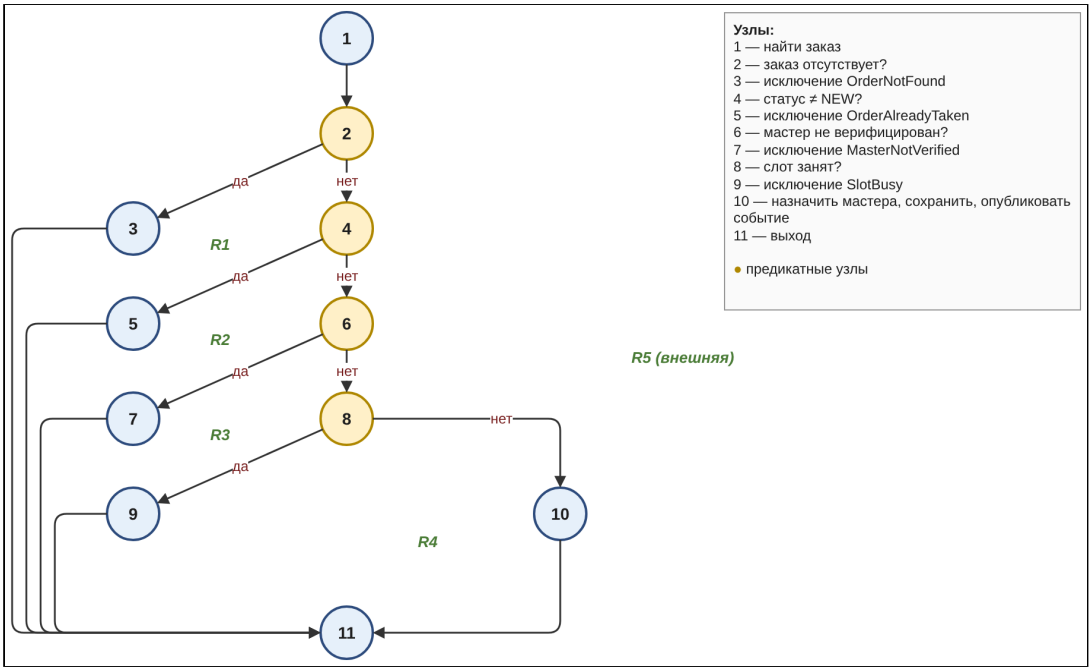


Рисунок 4.1 — Потоковый граф функции accept_order (draw.io)

Таблица 4.1 — Расчёт цикломатической сложности тремя способами

Способ	Расчёт	V(G)
1. По числу регионов	Плоский граф делит плоскость на 4 замкнутых региона R1–R4 и 1 внешний R5	**5**
2. По рёбрам и узлам: V(G) = E − N + 2	E = 14 рёбер (1-2, 2-3, 2-4, 3-11, 4-5, 4-6, 5-11, 6-7, 6-8, 7-11, 8-9, 8-10, 9-11, 10-11), N = 11 узлов: 14 − 11 + 2	**5**
3. По предикатным узлам: V(G) = P + 1	P = 4 (узлы 2, 4, 6, 8): 4 + 1	**5**

Все три способа дают V(G) = 5. Это верхняя граница числа независимых путей и минимальное число тестов, гарантирующее выполнение каждого оператора и каждой ветви хотя бы один раз.

Таблица 4.2 — Базисное множество независимых путей и тест-кейсы «белого ящика»

Путь	Узлы	Тест-кейс	Входные данные (состояние заглушек)	Ожидаемый результат
1	1-2-3-11	TC-W01	Заказа «X404» нет в репозитории	Исключение OrderNotFound; ничего не сохранено, событий нет
2	1-2-4-5-11	TC-W02	Заказ «O1» в статусе ACCEPTED, мастер M2	Исключение OrderAlreadyTaken; мастер заказа остаётся M2
3	1-2-4-6-7-11	TC-W03	Заказ «O1» NEW; мастер M1 не верифицирован	Исключение MasterNotVerified; статус остаётся NEW
4	1-2-4-6-8-9-11	TC-W04	Заказ «O1» NEW; M1 верифицирован; слот M1 на 12.11 15:00 занят	Исключение SlotBusy; заказ не сохранён
5	1-2-4-6-8-10-11	TC-W05	Заказ «O1» NEW; M1 верифицирован; слот свободен	Статус ACCEPTED, master_id = M1; заказ сохранён; слот зарезервирован; опубликовано событие OrderAccepted

Каждый следующий путь добавляет хотя бы одно новое ребро (3, 5, 7, 9, 10), поэтому пути независимы.

5. Код драйверов, заглушек и тестов, протоколы прогонов

Драйверы — тестовые функции pytest, которые вызывают компонент с подготовленными данными и проверяют результат. **Заглушки** (FakeOrderRepository, StubMasterDirectory, StubSchedule) заменяют БД, подсистему мастеров и график; **шпион** SpyEventBus запоминает опубликованные события, чтобы проверить взаимодействие с шиной INF-4 без Redis и Celery. Заглушки реализуют те же интерфейсы, что и реальные компоненты из ПР-6 (IOrderRepository, IEventBus), поэтому OrderService получает их через конструктор без изменения кода (внедрение зависимостей).

Листинг 5.1 — Компонент OrderService (xizmatgo/orders.py)

```
"""Компонент OrderService (SS-3): принятие заказа мастером, UC-08 / FR-08."""
from dataclasses import dataclass
from datetime import datetime
from typing import Optional, Protocol
from .states import OrderStatus


class OrderNotFound(Exception): ...
class OrderAlreadyTaken(Exception): ...
class MasterNotVerified(Exception): ...
class SlotBusy(Exception): ...


@dataclass
class Order:
    id: str
    scheduled_at: datetime
    status: OrderStatus = OrderStatus.NEW
    master_id: Optional[str] = None


class IOrderRepository(Protocol):
    def find_by_id(self, order_id: str) -> Optional[Order]: ...
    def save(self, order: Order) -> None: ...


class OrderService:
    def __init__(self, repo, masters, schedule, bus):
        self.repo, self.masters, self.schedule, self.bus = repo, masters, schedule, bus

    def accept_order(self, order_id: str, master_id: str) -> Order:
        order = self.repo.find_by_id(order_id)                      # 1
        if order is None:                                           # 2
            raise OrderNotFound(order_id)                           # 3
        if order.status != OrderStatus.NEW:                         # 4
            raise OrderAlreadyTaken(order_id)                       # 5
        if not self.masters.is_verified(master_id):                 # 6
            raise MasterNotVerified(master_id)                      # 7
        if self.schedule.is_busy(master_id, order.scheduled_at):    # 8
            raise SlotBusy(master_id)                               # 9
        order.master_id = master_id                                 # 10
        order.status = OrderStatus.ACCEPTED
        self.schedule.reserve(master_id, order.scheduled_at)
        self.repo.save(order)
        self.bus.publish({"type": "OrderAccepted", "order_id": order.id, "master_id": master_id})
        return order                                                # 11
```

Листинг 5.2 — Компонент PriceCalculator (xizmatgo/pricing.py, после исправления DEF-01)

```python
"""Компонент PriceCalculator (SS-3): расчёт предварительной стоимости, FR-04."""
from dataclasses import dataclass
import math

VISIT_FEE = 15_000          # выезд мастера, сум
URGENT_RATE = 0.30          # надбавка за срочность к стоимости работ
BULK_DISCOUNT = 0.10        # скидка на работы при объёме от 5 единиц
BULK_FROM = 5
QTY_MIN, QTY_MAX = 1, 10
HOUR_FROM, HOUR_TO = 8, 21  # начало визита: 08:00 … 20:59


class PriceError(ValueError):
    pass


@dataclass(frozen=True)
class PriceRange:
    min: int
    max: int


def calculate(base_price: int, quantity: int, hour: int, urgent: bool) -> PriceRange:
    if not isinstance(quantity, int) or isinstance(quantity, bool):
        raise PriceError("Объём должен быть целым числом")
    if quantity < QTY_MIN or quantity > QTY_MAX:
        raise PriceError(f"Объём должен быть от {QTY_MIN} до {QTY_MAX}")
    if not (HOUR_FROM <= hour < HOUR_TO):
        raise PriceError("Мастер приезжает с 08:00 до 21:00")
    if base_price <= 0:
        raise PriceError("Базовая цена должна быть больше нуля")
    work = base_price * quantity
    if quantity >= BULK_FROM:                    # скидка за объём (исправлено, DEF-01)
        work = work * (1 - BULK_DISCOUNT)
    if urgent:
        work = work * (1 + URGENT_RATE)
    total = round(work) + VISIT_FEE
    upper = math.ceil(total * 1.5 / 1000) * 1000  # верхняя граница диапазона, кратно 1000
    return PriceRange(total, upper)
```

Листинг 5.3 — Компонент OrderStateMachine (xizmatgo/states.py)

```python
"""Компонент OrderStateMachine (SS-3): допустимые переходы статусов заказа."""
from enum import Enum


class OrderStatus(str, Enum):
    NEW = "NEW"; ACCEPTED = "ACCEPTED"; ON_THE_WAY = "ON_THE_WAY"
    IN_PROGRESS = "IN_PROGRESS"; COMPLETED = "COMPLETED"; PAID = "PAID"; CANCELLED = "CANCELLED"


S = OrderStatus
TRANSITIONS = {   # (из, в): роли, которым разрешён переход
    (S.NEW, S.ACCEPTED): {"master"},
    (S.ACCEPTED, S.ON_THE_WAY): {"master"},
    (S.ON_THE_WAY, S.IN_PROGRESS): {"master"},
    (S.IN_PROGRESS, S.COMPLETED): {"master"},
    (S.COMPLETED, S.PAID): {"system"},
    (S.NEW, S.CANCELLED): {"client", "moderator"},
    (S.ACCEPTED, S.CANCELLED): {"client", "master", "moderator"},
    (S.ON_THE_WAY, S.CANCELLED): {"client", "moderator"},
}


def can_change(old: OrderStatus, new: OrderStatus, role: str) -> bool:
    return role in TRANSITIONS.get((old, new), set())
```

Листинг 5.4 — Заглушки и шпионы зависимостей (tests/conftest.py)

```python
"""Заглушки (stubs) и шпионы (spies) зависимостей OrderService."""
import sys, pathlib
sys.path.insert(0, str(pathlib.Path(__file__).resolve().parents[1]))
import pytest


class FakeOrderRepository:              # заглушка IOrderRepository (в памяти вместо PostgreSQL)
    def __init__(self, orders=()):
        self.data = {o.id: o for o in orders}
        self.saved = []
    def find_by_id(self, order_id):
        return self.data.get(order_id)
    def save(self, order):
        self.data[order.id] = order
        self.saved.append(order.id)


class StubMasterDirectory:              # заглушка подсистемы SS-1 / SS-6 (статус верификации)
    def __init__(self, verified=()):
        self.verified = set(verified)
    def is_verified(self, master_id):
        return master_id in self.verified


class StubSchedule:                     # заглушка графика мастера (СлотГрафика)
    def __init__(self, busy=()):
        self.busy = set(busy)
        self.reserved = []
    def is_busy(self, master_id, when):
        return (master_id, when) in self.busy
    def reserve(self, master_id, when):
        self.reserved.append((master_id, when))


class SpyEventBus:                      # шпион IEventBus (INF-4): запоминает события
    def __init__(self):
        self.events = []
    def publish(self, event):
        self.events.append(event)


@pytest.fixture
def bus():
    return SpyEventBus()
```

Листинг 5.5 — Тесты «чёрного ящика» PriceCalculator (tests/test_pricing.py)

```python
"""Тесты «чёрного ящика» PriceCalculator: классы эквивалентности и граничные значения."""
import pytest
from xizmatgo.pricing import calculate, PriceError, PriceRange

BASE = 80_000


@pytest.mark.parametrize("tc, qty, hour, urgent, expected", [
    ("TC-P01", 1, 8, False, PriceRange(95_000, 143_000)),     # нижняя граница объёма и часа
    ("TC-P02", 2, 12, False, PriceRange(175_000, 263_000)),   # типичное значение
    ("TC-P03", 4, 20, False, PriceRange(335_000, 503_000)),   # перед порогом скидки, верхняя граница часа
    ("TC-P04", 5, 12, False, PriceRange(375_000, 563_000)),   # порог скидки 10 %
    ("TC-P05", 10, 12, False, PriceRange(735_000, 1_103_000)),  # верхняя граница объёма
    ("TC-P06", 1, 12, True, PriceRange(119_000, 179_000)),    # срочно +30 %
    ("TC-P07", 5, 12, True, PriceRange(483_000, 725_000)),    # скидка и срочность вместе
])
def test_valid_classes(tc, qty, hour, urgent, expected):
    assert calculate(BASE, qty, hour, urgent) == expected


@pytest.mark.parametrize("tc, base, qty, hour", [
    ("TC-P08", BASE, 0, 12),      # объём ниже минимума
    ("TC-P09", BASE, 11, 12),     # объём выше максимума
    ("TC-P10", BASE, 2.5, 12),    # объём не целый
    ("TC-P11", BASE, 1, 7),       # час до 08:00
    ("TC-P12", BASE, 1, 21),      # час с 21:00
    ("TC-P13", 0, 1, 12),         # базовая цена не положительная
])
def test_invalid_classes(tc, base, qty, hour):
    with pytest.raises(PriceError):
        calculate(base, qty, hour, False)
```

Листинг 5.6 — Тесты «белого ящика» accept_order — драйверы (tests/test_accept_order.py)

```
"""Тесты «белого ящика» OrderService.accept_order: по одному на каждый базисный путь."""
from datetime import datetime
import pytest
from xizmatgo.orders import (OrderService, Order, OrderNotFound, OrderAlreadyTaken,
                             MasterNotVerified, SlotBusy)
from xizmatgo.states import OrderStatus
from conftest import FakeOrderRepository, StubMasterDirectory, StubSchedule

WHEN = datetime(2026, 11, 12, 15, 0)


def make_service(bus, orders=(), verified=("M1",), busy=()):
    repo = FakeOrderRepository(orders)
    sched = StubSchedule(busy)
    return OrderService(repo, StubMasterDirectory(verified), sched, bus), repo, sched


def test_path1_order_not_found(bus):                        # TC-W01: 1-2-3-11
    svc, repo, _ = make_service(bus)
    with pytest.raises(OrderNotFound):
        svc.accept_order("X404", "M1")
    assert bus.events == [] and repo.saved == []


def test_path2_order_already_taken(bus):                    # TC-W02: 1-2-4-5-11
    svc, repo, _ = make_service(bus, [Order("O1", WHEN, OrderStatus.ACCEPTED, "M2")])
    with pytest.raises(OrderAlreadyTaken):
        svc.accept_order("O1", "M1")
    assert repo.data["O1"].master_id == "M2"


def test_path3_master_not_verified(bus):                    # TC-W03: 1-2-4-6-7-11
    svc, repo, _ = make_service(bus, [Order("O1", WHEN)], verified=())
    with pytest.raises(MasterNotVerified):
        svc.accept_order("O1", "M1")
    assert repo.data["O1"].status == OrderStatus.NEW


def test_path4_slot_busy(bus):                              # TC-W04: 1-2-4-6-8-9-11
    svc, repo, _ = make_service(bus, [Order("O1", WHEN)], busy={("M1", WHEN)})
    with pytest.raises(SlotBusy):
        svc.accept_order("O1", "M1")
    assert repo.saved == []


def test_path5_success(bus):                                # TC-W05: 1-2-4-6-8-10-11
    svc, repo, sched = make_service(bus, [Order("O1", WHEN)])
    order = svc.accept_order("O1", "M1")
    assert order.status == OrderStatus.ACCEPTED and order.master_id == "M1"
    assert repo.saved == ["O1"] and sched.reserved == [("M1", WHEN)]
    assert bus.events == [{"type": "OrderAccepted", "order_id": "O1", "master_id": "M1"}]
```

Листинг 5.7 — Тесты OrderStateMachine (tests/test_states.py)

```
"""Тесты OrderStateMachine: допустимые и недопустимые переходы статусов."""
import pytest
from xizmatgo.states import can_change, OrderStatus as S


@pytest.mark.parametrize("tc, old, new, role, ok", [
    ("TC-S01", S.NEW, S.ACCEPTED, "master", True),
    ("TC-S02", S.IN_PROGRESS, S.COMPLETED, "master", True),
    ("TC-S03", S.COMPLETED, S.PAID, "system", True),
    ("TC-S04", S.ON_THE_WAY, S.CANCELLED, "client", True),
    ("TC-S05", S.NEW, S.ACCEPTED, "client", False),      # клиент не может «принять» заказ
    ("TC-S06", S.COMPLETED, S.PAID, "master", False),    # мастер не может отметить оплату сам
    ("TC-S07", S.IN_PROGRESS, S.CANCELLED, "client", False),  # отмена во время работы запрещена
    ("TC-S08", S.PAID, S.NEW, "moderator", False),       # возврат из конечного статуса
])
def test_transitions(tc, old, new, role, ok):
    assert can_change(old, new, role) is ok
```

5.1. Протокол прогона № 1

```
$ python -m pytest -v --tb=line tests
============================= test session starts ==============================
rootdir: /home/claude/pr11
plugins: platformdirs-4.13.0, anyio-4.15.1
collecting ... collected 26 items

tests/test_accept_order.py::test_path1_order_not_found PASSED            [  3%]
tests/test_accept_order.py::test_path2_order_already_taken PASSED        [  7%]
tests/test_accept_order.py::test_path3_master_not_verified PASSED        [ 11%]
tests/test_accept_order.py::test_path4_slot_busy PASSED                  [ 15%]
tests/test_accept_order.py::test_path5_success PASSED                    [ 19%]
tests/test_pricing.py::test_valid_classes[TC-P01-1-8-False-expected0] PASSED [ 23%]
tests/test_pricing.py::test_valid_classes[TC-P02-2-12-False-expected1] PASSED [ 26%]
tests/test_pricing.py::test_valid_classes[TC-P03-4-20-False-expected2] PASSED [ 30%]
tests/test_pricing.py::test_valid_classes[TC-P04-5-12-False-expected3] FAILED [ 34%]
tests/test_pricing.py::test_valid_classes[TC-P05-10-12-False-expected4] PASSED [ 38%]
tests/test_pricing.py::test_valid_classes[TC-P06-1-12-True-expected5] PASSED [ 42%]
tests/test_pricing.py::test_valid_classes[TC-P07-5-12-True-expected6] FAILED [ 46%]
tests/test_pricing.py::test_invalid_classes[TC-P08-80000-0-12] PASSED    [ 50%]
tests/test_pricing.py::test_invalid_classes[TC-P09-80000-11-12] PASSED   [ 53%]
tests/test_pricing.py::test_invalid_classes[TC-P10-80000-2.5-12] PASSED  [ 57%]
tests/test_pricing.py::test_invalid_classes[TC-P11-80000-1-7] PASSED     [ 61%]
tests/test_pricing.py::test_invalid_classes[TC-P12-80000-1-21] PASSED    [ 65%]
tests/test_pricing.py::test_invalid_classes[TC-P13-0-1-12] PASSED        [ 69%]
tests/test_states.py::test_transitions[TC-S01-NEW-ACCEPTED-master-True] PASSED [ 73%]
tests/test_states.py::test_transitions[TC-S02-IN_PROGRESS-COMPLETED-master-True] PASSED [ 76%]
tests/test_states.py::test_transitions[TC-S03-COMPLETED-PAID-system-True] PASSED [ 80%]
tests/test_states.py::test_transitions[TC-S04-ON_THE_WAY-CANCELLED-client-True] PASSED [ 84%]
tests/test_states.py::test_transitions[TC-S05-NEW-ACCEPTED-client-False] PASSED [ 88%]
tests/test_states.py::test_transitions[TC-S06-COMPLETED-PAID-master-False] PASSED [ 92%]
tests/test_states.py::test_transitions[TC-S07-IN_PROGRESS-CANCELLED-client-False] PASSED [ 96%]
tests/test_states.py::test_transitions[TC-S08-PAID-NEW-moderator-False] PASSED [100%]

================================== FAILURES ===================================
E   AssertionError: assert PriceRange(mi...0, max=623000) == PriceRange(mi...0, max=563000)

      Differing attributes:
      ['min', 'max']

      Drill down into differing attribute min:
        min: 415000 != 375000

      Drill down into differing attribute max:
        max: 623000 != 563000
/home/claude/pr11/tests/test_pricing.py:18: AssertionError: assert PriceRange(mi...0, max=623000) ==
PriceRange(mi...0, max=563000)
E   AssertionError: assert PriceRange(mi...0, max=803000) == PriceRange(mi...0, max=725000)

      Differing attributes:
      ['min', 'max']

      Drill down into differing attribute min:
        min: 535000 != 483000

      Drill down into differing attribute max:
        max: 803000 != 725000
/home/claude/pr11/tests/test_pricing.py:18: AssertionError: assert PriceRange(mi...0, max=803000) ==
PriceRange(mi...0, max=725000)
=========================== short test summary info ============================
FAILED tests/test_pricing.py::test_valid_classes[TC-P04-5-12-False-expected3]
FAILED tests/test_pricing.py::test_valid_classes[TC-P07-5-12-True-expected6]
======================== 2 failed, 24 passed in 0.04s =========================
```

5.2. Протокол прогона № 2 (после исправления DEF-01)

```
$ python -m pytest -v tests
============================= test session starts ==============================
rootdir: /home/claude/pr11
plugins: platformdirs-4.13.0, anyio-4.15.1
collecting ... collected 26 items

tests/test_accept_order.py::test_path1_order_not_found PASSED            [  3%]
tests/test_accept_order.py::test_path2_order_already_taken PASSED        [  7%]
tests/test_accept_order.py::test_path3_master_not_verified PASSED        [ 11%]
tests/test_accept_order.py::test_path4_slot_busy PASSED                  [ 15%]
tests/test_accept_order.py::test_path5_success PASSED                    [ 19%]
tests/test_pricing.py::test_valid_classes[TC-P01-1-8-False-expected0] PASSED [ 23%]
tests/test_pricing.py::test_valid_classes[TC-P02-2-12-False-expected1] PASSED [ 26%]
tests/test_pricing.py::test_valid_classes[TC-P03-4-20-False-expected2] PASSED [ 30%]
tests/test_pricing.py::test_valid_classes[TC-P04-5-12-False-expected3] PASSED [ 34%]
tests/test_pricing.py::test_valid_classes[TC-P05-10-12-False-expected4] PASSED [ 38%]
tests/test_pricing.py::test_valid_classes[TC-P06-1-12-True-expected5] PASSED [ 42%]
tests/test_pricing.py::test_valid_classes[TC-P07-5-12-True-expected6] PASSED [ 46%]
tests/test_pricing.py::test_invalid_classes[TC-P08-80000-0-12] PASSED    [ 50%]
```

```
tests/test_pricing.py::test_invalid_classes[TC-P09-80000-11-12] PASSED   [ 53%]
tests/test_pricing.py::test_invalid_classes[TC-P10-80000-2.5-12] PASSED  [ 57%]
tests/test_pricing.py::test_invalid_classes[TC-P11-80000-1-7] PASSED     [ 61%]
tests/test_pricing.py::test_invalid_classes[TC-P12-80000-1-21] PASSED    [ 65%]
tests/test_pricing.py::test_invalid_classes[TC-P13-0-1-12] PASSED        [ 69%]
tests/test_states.py::test_transitions[TC-S01-NEW-ACCEPTED-master-True] PASSED [ 73%]
tests/test_states.py::test_transitions[TC-S02-IN_PROGRESS-COMPLETED-master-True] PASSED [ 76%]
tests/test_states.py::test_transitions[TC-S03-COMPLETED-PAID-system-True] PASSED [ 80%]
tests/test_states.py::test_transitions[TC-S04-ON_THE_WAY-CANCELLED-client-True] PASSED [ 84%]
tests/test_states.py::test_transitions[TC-S05-NEW-ACCEPTED-client-False] PASSED [ 88%]
tests/test_states.py::test_transitions[TC-S06-COMPLETED-PAID-master-False] PASSED [ 92%]
tests/test_states.py::test_transitions[TC-S07-IN_PROGRESS-CANCELLED-client-False] PASSED [ 96%]
tests/test_states.py::test_transitions[TC-S08-PAID-NEW-moderator-False] PASSED [100%]

============================== 26 passed in 0.04s ==============================
```

Таблица 5.1 — Сводный протокол тестирования

Показатель	Прогон № 1 (10.11.2026, 13:40)	Прогон № 2 (10.11.2026, 13:52)
Версия	0.1 (коммит a1f3c9e)	0.1.1 (коммит 7b2d04f — исправление DEF-01)
Выполнено тестов	26	26
Пройдено	24	**26**
Не пройдено	**2 (TC-P04, TC-P07)**	0
K-1 PriceCalculator	11 / 13	13 / 13
K-2 accept_order	5 / 5	5 / 5
K-3 OrderStateMachine	8 / 8	8 / 8
Найдено дефектов	1 (DEF-01)	0 новых
Исполнитель	Ким Д. (QA); окружение: Python 3.13, pytest 9.1, Linux	

Номера коммитов и исполнитель условные (учебный проект); вывод pytest — реальный результат запуска приведённого кода.

6. Отчёт о дефекте и матрица трассируемости

Таблица 6.1 — Отчёт о дефекте DEF-01

ID / заголовок	**DEF-01**. Скидка 10 % не применяется при объёме ровно 5 единиц
Компонент / версия	K-1 PriceCalculator (xizmatgo/pricing.py), версия 0.1
Обнаружен тестами	TC-P04, TC-P07 (граничное значение q = 5), прогон № 1
Серьёзность / приоритет	S2 — высокая (неверная цена для клиента: завышение на 40 000–52 000 сум; нарушает FR-04 и QR-03) / высокий
Шаги воспроизведения	1. Вызвать calculate(base_price = 80 000, quantity = 5, hour = 12, urgent = False). 2. Сравнить результат с ожидаемым.
Ожидаемый результат	PriceRange(min = 375 000, max = 563 000) — скидка 10 % на работы «от 5 единиц» по FR-04
Фактический результат	PriceRange(min = 415 000, max = 623 000) — скидка не применена. Для urgent = True: 535 000 вместо 483 000
Причина	Строка 33: строгое сравнение `if quantity > BULK_FROM` вместо `>=` — типичная ошибка «на единицу» на границе класса эквивалентности
Исправление	`if quantity >= BULK_FROM` (коммит 7b2d04f), исполнитель Рахимов Ж.; код-ревью — Ким Д.
Проверка исправления	Прогон № 2: TC-P04 и TC-P07 пройдены; регрессия — все 26 тестов пройдены
Статус	Новый → Подтверждён → Исправлен → Проверен → **Закрыт** (10.11.2026)

Таблица 6.2 — Матрица трассируемости «требование — тест-кейс»

Требование	Содержание	Тест-кейсы	Статус (прогон № 2)
FR-04 / UC-03.1	Формула цены: база × объём + выезд	TC-P01, TC-P02, TC-P03	**пройдены**
FR-04 (скидка)	Скидка 10 % от 5 единиц	TC-P04, TC-P05, TC-P07	**пройдены (после DEF-01)**
FR-04 (срочность)	Надбавка 30 % при «Срочно»	TC-P06, TC-P07	**пройдены**
FR-04 (ограничения)	Объём 1–10, целый; визит 08:00–21:00; цена > 0	TC-P08…TC-P13	**пройдены**
QR-03 (ПР-9)	100 % эталонных расчётов цены верны	TC-P01…TC-P07	**7 / 7 = 100 %**
FR-08 / UC-08	Принять можно только существующий заказ NEW	TC-W01, TC-W02	**пройдены**
FR-11 / UC-10	Неверифицированный мастер не принимает заказы	TC-W03	**пройден**
FR-08 / UC-08	Время мастера не должно быть занято; при принятии слот блокируется	TC-W04, TC-W05	**пройдены**
FR-07	Смена статуса и событие для уведомления клиента	TC-W05, TC-S01…TC-S04	**пройдены**
FR-07 / SR-04 (ПР-10)	Недопустимые переходы и роли запрещены	TC-S05…TC-S08	**пройдены**

Каждое требование связано хотя бы с одним тест-кейсом; все 26 тест-кейсов пройдены, критерии завершения тест-плана выполнены.

7. Вывод о проделанной работе

В практической работе по варианту № 8 «Разработка умного сайта для заказа услуг» выполнено компонентное тестирование подсистемы SS-3 «Управление заказами» системы «XizmatGo»:

1. Выбраны 3 компонента: PriceCalculator, OrderService.accept_order, OrderStateMachine. Составлен тест-план: объекты и проверяемые требования, подход, окружение и средства, критерии начала, завершения и приостановки, роли, сроки и риски.
2. Для PriceCalculator выделены 6 допустимых и 6 недопустимых классов эквивалентности и граничные значения по 4 входным условиям. Разработаны 13 тест-кейсов «чёрного ящика» с рассчитанными ожидаемыми результатами.
3. Для функции accept_order с 4 условиями в draw.io построен потоковый граф. Цикломатическая сложность вычислена тремя способами (регионы, E − N + 2, P + 1) — V(G) = 5. Определены 5 базисных путей и тест-кейс для каждого.
4. Тест-кейсы реализованы 26 автоматизированными тестами pytest с драйверами, заглушками и шпионом зависимостей. Первый прогон дал 24 пройденных и 2 непройденных теста.
5. Оформлен отчёт о дефекте DEF-01: на граничном значении объёма 5 скидка не применялась из-за ошибки «на единицу». После исправления повторный прогон всех тестов — 26 из 26 пройдены. Составлена матрица трассируемости «требование — тест-кейс».